UT1 · KOTLIN FUNDAMENTALS

Block 5

Classes and objects

What you will learn
Declare classes with a **primary constructor** and properties in a single line
Use properties instead of getters and setters, and protect them with `private set`
Validate data with `init` and `require`
Write the Animal family in Kotlin: `open`, `override`, abstract classes and interfaces
Use polymorphism with `is` and smart casts
Use the special classes: `data class`, `enum class` and `object`
Add functions to existing types with **extension functions**

You already know object-oriented programming: in UT0 you worked with classes, constructors, encapsulation, inheritance, abstract classes, interfaces and polymorphism in Java. **None of these concepts is new.** This block shows how to write them in Kotlin, which needs much less code, and introduces three special kinds of class that Kotlin adds and that you will use constantly in Jetpack Compose.

To make the comparison easy, we will use the same **Animal family** you studied in UT0: `Animal`, `Dog`, `Cat`, `Hippo` and the `Pet` interface.

Contenido

1. Your first Kotlin class 3
1.1 Creating objects 3
1.2 Default values and named arguments 4
2. Properties 4
2.1 Computed properties 4
2.2 Protecting properties: private set 5
3. Validating data: init and require 5
4. Methods and toString 6
5. Visibility modifiers 6
6. Inheritance: open and override 7
6.1 Calling the parent version: super 8
7. Abstract classes 8
8. Interfaces 9
9. Polymorphism, is and smart casts 10
9.1 Checking the type with is 10
10. Data classes 11
10.1 Data classes and collections 11
11. Enum classes 12
11.1 Enums with properties 13
12. object and companion object 13
12.1 companion object: the replacement for static 13
13. Extension functions 14
14. Summary: Java vs Kotlin 15
15. Exercises 15
Exercise 1 · From Java to Kotlin 15
Exercise 2 · Bank account 16
Exercise 3 · The Animal family 16
Exercise 4 · Product catalogue with data classes 17
Exercise 5 · Order status 17
Exercise 6 · Extension functions 17
Challenge · Geometric shapes in Kotlin 17

1. Your first Kotlin class

This is a typical Java class with two attributes, a constructor and its getters and setters. Compare it with the Kotlin version, which does exactly the same:

Java

```
public class Animal {
    private String name;
    private int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}
```

Kotlin

```
class Animal(val name: String,
             var age: Int)
```

One line replaces sixteen. The parentheses after the class name are the **primary constructor**, and writing `val` or `var` before each parameter turns it into a **property** of the class:

- `val name` is a read-only property: it has a getter, but no setter.
- `var age` is a mutable property: it has a getter and a setter.

1.1 Creating objects

```
class Animal(val name: String, var age: Int)

fun main() {
    val rex = Animal("Rex", 3)       // no "new" keyword
    println(rex.name)                // Rex
    rex.age = 4                      // calls the setter
    println(rex.age)                 // 4
    // rex.name = "Max"              // ERROR: val cannot be reassigned
}
```

COMING FROM JAVA · No new, no getters

Kotlin does not use the keyword `new`: you create an object by calling the class as if it were a function, `Animal("Rex", 3)`.

You write `rex.name` and `rex.age = 4` instead of `rex.getName()` and `rex.setAge(4)`. Behind the scenes Kotlin still generates the getter and setter, so Java code can use the class normally.

1.2 Default values and named arguments

Constructor parameters work exactly like function parameters (Block 3): they can have default values and can be passed by name. This makes Java's overloaded constructors unnecessary:

```
class Animal(val name: String, var age: Int = 0, val species: String = "Unknown")

fun main() {
    val a = Animal("Rex")
    val b = Animal("Tom", 2)
    val c = Animal("Gloria", species = "Hippo")   // skips age
    println("${a.age} ${b.age} ${c.species}")   // 0 2 Hippo
}
```

2. Properties

Properties do not have to come from the constructor. They can also be declared in the body of the class, with an initial value:

```
class Dog(val name: String) {
    var energy = 100          // every new dog starts with 100
    val legs = 4
}

fun main() {
    val rex = Dog("Rex")
    rex.energy -= 30
    println("${rex.name}: ${rex.energy} energy, ${rex.legs} legs")
    // Rex: 70 energy, 4 legs
}
```

2.1 Computed properties

A property can calculate its value every time it is read, with a custom **getter**. It looks like a property from outside, but works like a function without parameters:

```
class Animal(val name: String, var age: Int) {
    val isAdult: Boolean
        get() = age >= 2
}

fun main() {
    val tom = Animal("Tom", 1)
    println(tom.isAdult)   // false
    tom.age = 3
    println(tom.isAdult)   // true (calculated again)
}
```

2.2 Protecting properties: private set

Encapsulation means that an object controls how its data changes. If a property must be readable from outside but only modifiable from inside the class, use `private set`:

```
class Dog(val name: String) {
    var energy = 100
        private set     // only the class can change it

    fun play() {
        energy -= 20
    }
}

fun main() {
    val rex = Dog("Rex")
    rex.play()
    println(rex.energy)   // 80
    // rex.energy = 500   // ERROR: the setter is private
}
```

This is the Kotlin equivalent of a Java class with a public getter and no public setter: the only way to change `energy` is through the methods of the class, which decide the rules.

3. Validating data: init and require

The primary constructor has no body, so where do we check that the data is valid? In an `init` **block**, which runs every time an object is created, just after the constructor:

```
class Animal(val name: String, var age: Int) {
    init {
        require(name.isNotBlank()) { "The name cannot be empty" }
        require(age >= 0) { "The age cannot be negative" }
        println("Animal created: $name")
    }
}

fun main() {
    val rex = Animal("Rex", 3)      // Animal created: Rex
    val ghost = Animal("Ghost", -1) // crashes: IllegalArgumentException
}
```

`require(condition) { message }` stops the program with an `IllegalArgumentException` and that message if the condition is false. It is the standard way to say "this object cannot exist with these values". It is better to fail immediately, when the object is created, than to have an animal with age -1 causing strange errors later.

> **TIP · Secondary constructors**
> Kotlin also has secondary constructors (`constructor(...)` inside the class body), but you will rarely need them: default values in the primary constructor solve almost every case that needs several constructors in Java.

4. Methods and toString

Functions declared inside a class are its **methods** (in Kotlin they are also called **member functions**). They can use the properties of the class directly; `this` is optional, as in Java:

```
class Dog(val name: String, var age: Int) {
    fun bark() {
        println("$name says Woof!")
    }

    fun haveBirthday() {
        age++
        println("Happy birthday, $name! You are now $age.")
    }

    override fun toString(): String = "Dog(name=$name, age=$age)"
}

fun main() {
    val rex = Dog("Rex", 3)
    rex.bark()            // Rex says Woof!
    rex.haveBirthday()    // Happy birthday, Rex! You are now 4.
    println(rex)          // Dog(name=Rex, age=4)
}
```

Every class inherits `toString()` from `Any`, the Kotlin equivalent of Java's `Object`. By default it prints something like `Dog@1b6d3586`, which is not useful; overriding it lets `println(rex)` show meaningful information. Notice that `override` is a keyword in Kotlin, not an annotation like Java's `@Override`, and it is **mandatory**.

5. Visibility modifiers

Modifier	Visible from…	Difference with Java
`public`	Everywhere	**It is the default**: you do not need to write it
`private`	Only inside the class (or the file, for top-level declarations)	Same idea
`protected`	The class and its subclasses	Not visible from the rest of the package, unlike Java
`internal`	Anywhere in the same module (e.g. the same IntelliJ module or app)	Does not exist in Java

Java has a default "package-private" visibility that Kotlin does not have. In Kotlin, if you write nothing, everything is public, so remember to mark as `private` whatever should not be touched from outside, for example with `private val` in the constructor: `class Account(private val pin: String)`.

6. Inheritance: open and override

In Kotlin, classes and methods are **final by default**: they cannot be inherited or overridden unless you allow it explicitly with `open`. This is the opposite of Java, where everything can be extended unless it is marked `final`.

```
open class Animal(val name: String) {
    open fun makeNoise() {
        println("$name makes a noise.")
    }
    fun sleep() {                 // not open: subclasses cannot change it
        println("$name sleeps.")
    }
}

class Dog(name: String) : Animal(name) {
    override fun makeNoise() {
        println("$name: Woof!")
    }
}

fun main() {
    val rex = Dog("Rex")
    rex.makeNoise()   // Rex: Woof!
    rex.sleep()       // Rex sleeps. (inherited)
}
```

Look at the class header: `class Dog(name: String) : Animal(name)`.

- The colon `:` replaces Java's `extends`.
- `Animal(name)` calls the constructor of the parent class, like `super(name)` in Java.
- `name` has **no** `val` in `Dog`: it is just a parameter passed to `Animal`, which already declares the property. Writing `val name` again does not compile: the compiler says that it "hides member of supertype Animal".

Java

```
public class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }
    @Override
    public void makeNoise() {
        System.out.println(
            name + ": Woof!");
    }
}
```

Kotlin

```
class Dog(name: String) : Animal(name) {
    override fun makeNoise() {
        println("$name: Woof!")
    }
}
```

COMING FROM JAVA · Why final by default?

Designing a class to be extended safely takes thought. Kotlin makes you decide on purpose which classes and methods can change in subclasses (`open`), so nobody breaks a class by accident. If you forget `open`, the compiler reminds you: "This type is final, so it cannot be extended".

6.1 Calling the parent version: super

```
open class Animal(val name: String) {
    open fun eat() {
        println("$name eats.")
    }
}

class Hippo(name: String) : Animal(name) {
    override fun eat() {
        super.eat()                        // first, what every animal does
        println("$name eats 40 kg of grass.")  // then, its own part
    }
}

fun main() {
    Hippo("Gloria").eat()
}
```

7. Abstract classes

As in Java, an **abstract class** cannot be instantiated and can declare **abstract methods** without a body, which every subclass must implement. Abstract classes and abstract members are `open` automatically, so you do not need to write `open` on them.

```
abstract class Animal(val name: String) {
    abstract fun makeNoise()          // no body: each animal decides

    open fun eat() {                  // has a body, may be overridden
        println("$name eats.")
    }

    fun sleep() {                     // has a body, cannot be overridden
        println("$name sleeps.")
    }
}

class Cat(name: String) : Animal(name) {
    override fun makeNoise() {
        println("$name: Meow!")
    }
}

fun main() {
    val tom = Cat("Tom")
    tom.makeNoise()   // Tom: Meow!
    tom.eat()         // Tom eats.
    // val a = Animal("?")   // ERROR: Animal is abstract
}
```

Member of an abstract class	Keyword	Must subclasses override it?
Without body	`abstract fun`	Yes, always
With body, changeable	`open fun`	Only if they want to
With body, fixed	`fun`	They cannot

8. Interfaces

An **interface** defines a set of methods that a class promises to implement. In Kotlin, a class implements interfaces with the same colon used for inheritance, separated by commas. Interfaces can also include methods **with a default body**.

```
interface Pet {
    fun beFriendly()                  // must be implemented
    fun play() {                      // default implementation
        println("Playing...")
    }
}

class Dog(name: String) : Animal(name), Pet {
    override fun makeNoise() { println("$name: Woof!") }
    override fun beFriendly() { println("$name wags its tail.") }
    override fun play() { println("$name runs after the ball.") }
}

class Cat(name: String) : Animal(name), Pet {
    override fun makeNoise() { println("$name: Meow!") }
    override fun beFriendly() { println("$name purrs.") }
    // play() not overridden: uses the default one
}

fun main() {
    val rex = Dog("Rex")
    val tom = Cat("Tom")
    rex.play()          // Rex runs after the ball.
    tom.play()          // Playing...
    tom.beFriendly()    // Tom purrs.
}
```

Java

```
public class Dog extends Animal
        implements Pet { … }
```

Kotlin

```
class Dog(name: String) :
    Animal(name), Pet { … }
```

Notice the difference in the header: the parent class is written with parentheses, `Animal(name)`, because its constructor is called; the interface is written without them, `Pet`, because interfaces have no constructors.

9. Polymorphism, is and smart casts

Polymorphism works exactly as in Java: a variable or a list of type `Animal` can hold any subclass, and each object runs **its own** version of the overridden methods.

```
fun main() {
    val animals: List<Animal> = listOf(Dog("Rex"), Cat("Tom"), Hippo("Gloria"))

    for (animal in animals) {
        animal.makeNoise()    // each one makes its own noise
    }
}
```

Output

```
Rex: Woof!
Tom: Meow!
Gloria: Grunt!
```

9.1 Checking the type with is

Kotlin replaces Java's `instanceof` with `is`. And, as with null checks in Block 4, after checking the type Kotlin applies a **smart cast**: there is no need to write the cast.

```
fun describe(animal: Animal) {
    if (animal is Pet) {
        animal.beFriendly()            // smart cast: animal is a Pet here
    } else {
        println("${animal.name} is not a pet.")
    }
}

fun main() {
    describe(Dog("Rex"))       // Rex wags its tail.
    describe(Hippo("Gloria"))  // Gloria is not a pet.
}
```

Java

```
if (animal instanceof Pet) {
    Pet pet = (Pet) animal;
    pet.beFriendly();
}
```

Kotlin

```
if (animal is Pet) {
    animal.beFriendly()
}
```

`is` also works in the branches of a `when`, which is a very clean way to act differently depending on the type:

```
fun favouriteFood(animal: Animal): String = when (animal) {
    is Dog -> "Bones"
    is Cat -> "Fish"
    else -> "Grass"
}

fun main() {
    println(favouriteFood(Cat("Tom")))   // Fish
}
```

TIP · Lists and types

Collection functions also understand types: `animals.filterIsInstance<Pet>()` returns a `List<Pet>` with only the animals that are pets, and `animals.firstOrNull { it is Pet }` returns the first one, or `null` if there is none.

10. Data classes

Many classes exist only to **hold data**: a product, a contact, a message. For them, Kotlin offers the `data class`. Just add `data` before `class`, and the compiler automatically generates `toString()`, `equals()`, `hashCode()` and a `copy()` function.

```
data class Product(val name: String, val price: Double, val stock: Int = 0)

fun main() {
    val keyboard = Product("Keyboard", 25.0, 10)

    println(keyboard)
    // Product(name=Keyboard, price=25.0, stock=10)

    val sameKeyboard = Product("Keyboard", 25.0, 10)
    println(keyboard == sameKeyboard)     // true: compares the content

    val cheaper = keyboard.copy(price = 19.99)   // same data, different price
    println(cheaper)
    // Product(name=Keyboard, price=19.99, stock=10)

    val (name, price) = keyboard          // destructuring
    println("$name costs $price")         // Keyboard costs 25.0
}
```

With a normal class, `println(keyboard)` would show something like `Product@6d06d69c`, and `==` would compare whether both variables point to **the same object**, so it would be `false`. A data class compares the **content**, like `String`.

10.1 Data classes and collections

In Block 4 we stored products in a `Map<String, Double>`. With a data class, each product can have all the information it needs, and lists of objects work with all the functions you already know:

```
data class Product(val name: String, val price: Double, val stock: Int = 0)

fun main() {
    val catalogue = listOf(
        Product("Keyboard", 25.0, 10),
        Product("Mouse", 12.5, 0),
        Product("Monitor", 149.99, 3)
    )

    val available = catalogue.filter { it.stock > 0 }
    val names = available.map { it.name }
    val mostExpensive = catalogue.maxByOrNull { it.price }
    val stockValue = catalogue.sumOf { it.price * it.stock }

    println(names)                          // [Keyboard, Monitor]
    println(mostExpensive?.name)            // Monitor
    println("%.2f".format(stockValue))      // 699.97
}
```

TIP · Data classes in Compose

In Compose, the information shown on a screen is usually stored in data classes, and `copy()` is the standard way to create a modified version of that information. Use `val` in their properties: data classes work best when their objects do not change.

11. Enum classes

An **enum class** represents a fixed set of possible values: the days of the week, the states of an order, the sizes of a T-shirt. Using an enum instead of a `String` prevents typos such as `"SHIPED"`, because the compiler knows all the valid values.

```
enum class OrderStatus {
    PENDING, PAID, SHIPPED, DELIVERED
}

fun describe(status: OrderStatus): String = when (status) {
    OrderStatus.PENDING -> "Waiting for payment"
    OrderStatus.PAID -> "Preparing your order"
    OrderStatus.SHIPPED -> "On its way"
    OrderStatus.DELIVERED -> "Enjoy it!"
}

fun main() {
    val status = OrderStatus.SHIPPED
    println(describe(status))          // On its way
    println(status.name)               // SHIPPED
    println(OrderStatus.entries)       // [PENDING, PAID, SHIPPED, DELIVERED]
}
```

Notice that the `when` has **no** `else`: because the compiler knows every possible value of the enum, it can check that all of them are covered. If you add a new status, such as `CANCELLED`,

every `when` that forgets it stops compiling, so you cannot forget to handle it. These are the "special types" that Block 2 mentioned.

11.1 Enums with properties

```
enum class Size(val label: String, val extraPrice: Double) {
    SMALL("S", 0.0),
    MEDIUM("M", 1.5),
    LARGE("L", 3.0)
}

fun main() {
    val size = Size.LARGE
    println("Size ${size.label}: +${size.extraPrice} €")   // Size L: +3.0 €
}
```

12. object and companion object

Sometimes a program needs **exactly one** instance of something: the settings of the app, a counter, a logger. In Java you would write a Singleton class. In Kotlin, `object` declares a class and its only instance at the same time:

```
object ShopSettings {
    const val TAX_PERCENTAGE = 21.0
    var currency = "€"

    fun formatPrice(price: Double) = "%.2f %s".format(price, currency)
}

fun main() {
    println(ShopSettings.formatPrice(9.5))   // 9,50 € or 9.50 €
    ShopSettings.currency = "$"
    println(ShopSettings.formatPrice(9.5))   // 9,50 $ or 9.50 $
}
```

An `object` is used by its name, without parentheses: you never create it with `ShopSettings()`.

12.1 companion object: the replacement for static

Kotlin has no `static` keyword. Members that belong to the **class** instead of to each object go inside a `companion object`, and are used with the name of the class, exactly like static members in Java:

```
class Dog(val name: String) {
    companion object {
        const val MAX_AGE = 20
        var dogsCreated = 0
    }

    init {
        dogsCreated++
    }
}

fun main() {
    Dog("Rex")
    Dog("Toby")
    println(Dog.dogsCreated)   // 2
    println(Dog.MAX_AGE)       // 20
}
```

13. Extension functions

An **extension function** adds a new function to an existing type, **without modifying it or inheriting from it**, even to types you did not write, such as `String`, `Int` or `Double`. Write the type, a dot and the name of the function; inside, `this` is the value the function is called on:

```
fun String.initial(): Char = this.firstOrNull()?.uppercaseChar() ?: '?'

fun Double.toEuros(): String = "%.2f €".format(this)

fun Int.isEven(): Boolean = this % 2 == 0

fun main() {
    println("laura".initial())   // L
    println(19.5.toEuros())      // 19,50 € or 19.50 €
    println(7.isEven())          // false
}
```

They are called exactly like normal methods, with a dot. This is how many functions you have already used are written, for example `let` from Block 4, and the reason why you can write `name.isNullOrBlank()` even if `name` is `null`.

> **WATCH OUT · Use them with care**
> Extension functions are great for small, general helpers (formatting a price, validating an email). They do not have access to `private` members of the class, and if you create too many, nobody will know where a function comes from. Keep them in a file with a clear name, such as `Extensions.kt`.

14. Summary: Java vs Kotlin

Concept	Java	Kotlin
Class with attributes	Fields + constructor + getters/setters	`class Animal(val name: String, var age: Int)`
Create an object	`new Animal("Rex", 3)`	`Animal("Rex", 3)`
Read / change a property	`a.getAge()`, `a.setAge(4)`	`a.age`, `a.age = 4`
Read-only property	Getter, no setter	`val`
Default visibility	Package-private	`public`
Inheritance	`extends Animal` + `super(name)`	`: Animal(name)`
Can be extended…	Unless it is `final`	Only if it is `open` or `abstract`
Override	`@Override` (optional)	`override` (mandatory)
Interfaces	`implements Pet`	`, Pet`
Type check	`instanceof` + cast	`is` + smart cast
Data holder	Write `toString`, `equals`, `hashCode`…	`data class`
Singleton	Private constructor + static instance	`object`
Static members	`static`	`companion object`

15. Exercises

Create the files for this block in a new package, `exercises.classes`. Classes usually go in their own file, named like the class (`Animal.kt`, `Dog.kt`…), and `main` in a separate file.

Exercise 1 · From Java to Kotlin

Translate this Java class into Kotlin, writing as little code as possible. Then create two students from `main`, change the mark of one of them and print both.

```java
public class Student {
    private String name;
    private String group;
    private double mark;

    public Student(String name, String group, double mark) {
        this.name = name;
        this.group = group;
        this.mark = mark;
    }
    public Student(String name) {
        this(name, "2DAM", 0.0);
    }
    public String getName() { return name; }
    public String getGroup() { return group; }
    public double getMark() { return mark; }
    public void setMark(double mark) { this.mark = mark; }
    public boolean hasPassed() { return mark >= 5; }
}
```

Hints: the second constructor can be replaced with default values; `hasPassed()` can become a computed property `passed`. Which properties should be `val`, and which `var`?

Exercise 2 · Bank account

Write the class `BankAccount(val owner: String)` with:

- A property `balance`, starting at 0.0, that can be read from outside but **not** modified directly (`private set`).
- `deposit(amount: Double)`: adds the amount. Use `require` so that the amount is greater than 0.
- `withdraw(amount: Double): Boolean`: subtracts the amount only if there is enough balance, and returns whether it was possible.
- An `init` block that checks that the owner's name is not blank, and a `toString()` that shows the owner and the balance with two decimals.

Test it from `main`, including a withdrawal that is not possible. What happens if you try `account.balance = 1000.0`?

Exercise 3 · The Animal family

1. Write the abstract class `Animal(val name: String)` with the abstract method `makeNoise()`, the open method `eat()` and the normal method `sleep()`.
2. Write the subclasses `Dog`, `Cat`, `Wolf` and `Hippo`, each with its own noise. `Hippo` must override `eat()` and call `super.eat()`.
3. Write the interface `Pet` with `beFriendly()` and a default `play()`. Only `Dog` and `Cat` are pets.
4. In `main`, create a `List<Animal>` with one animal of each type. Make all of them make a noise, then make only the pets play, using `is` (or `filterIsInstance`).

Exercise 4 · Product catalogue with data classes

Rewrite **Exercise 6 of Block 4** (Manage a product list) using `data class Product(val name: String, val price: Double, val stock: Int)` and a `MutableList<Product>` instead of a map. Implement at least: show the catalogue, add a product, find a product by name (`find` + `?:`), show the products with stock, show the most expensive product, and apply a 10 % discount to a product using `copy()`.

Think about it: which version is easier to extend if tomorrow each product also needs a category: the map or the data class?

Exercise 5 · Order status

Write the enum `OrderStatus` with `PENDING`, `PAID`, `SHIPPED`, `DELIVERED` and `CANCELLED`, where each value has a property `description: String`. Write the function `nextStatus(status: OrderStatus): OrderStatus?`, which returns the next status in the normal flow, or `null` for `DELIVERED` and `CANCELLED`. Use a `when` **without** `else`. In `main`, follow an order from `PENDING` until there is no next status, printing each description.

Exercise 6 · Extension functions

Write these extension functions in a file called `Extensions.kt` and test them from `main`:

- `Double.toEuros(): String` → `19.5.toEuros()` returns `"19,50 €"` (or `"19.50 €"`).
- `String.isValidEmail(): Boolean` → true if it contains exactly one `@` and at least one `.` after it.
- `String.capitalizeWords(): String` → `"hello kotlin world"` returns `"Hello Kotlin World"` (hint: `split(" ")`, `map` and `joinToString(" ")`).
- `List<Double>.average2(): String` → the average of the list formatted with two decimals, or `"No data"` if the list is empty.

Challenge · Geometric shapes in Kotlin

Migrate the **FigurasGeometricas** project from UT0 (in the repository) to Kotlin, in a new package `exercises.classes.shapes`:

- `Figura` as an abstract class with the abstract method `calcularArea(): Double` and the method `mostrarInfo()`.
- `Circulo`, `Rectangulo` and `Triangulo` as subclasses, and the interface `Coloreable` with `pintar(color: String)`, implemented only by `Circulo` and `Rectangulo`.
- `GestorFiguras` as an `object`. Its function `buscarPrimeraColoreable` must receive a `List<Figura>` and return `Coloreable?` (no `instanceof`, no casts, no `null` without a `?` in the type).
- Reproduce both cases of the original `Main.java`, and handle the result of `buscarPrimeraColoreable` with the tools of Block 4.

Finally, compare the number of lines of both versions. Which Kotlin features made the biggest difference?